C++ PROGRAMMING TEXTBOOK NOTES

Module: Generic Programming & C++ Templates

Author: Gaurav Kandpal | **Platform:** CS-Simplified | **Level:** Beginner to Advanced

1. INTRODUCTION TO C++ TEMPLATES & GENERIC PROGRAMMING

A **Template** in C++ allows us to write a single piece of code (function or class) that works seamlessly with **multiple data types** without code duplication. Templates form the backbone of **Generic Programming** and the C++ Standard Template Library (STL).

Instead of writing separate function overloads or classes for int, double, float, or custom objects, a template uses **type parameters** (placeholders) that are substituted at compile time.

Core Advantages of C++ Templates:

- **Code Reusability:** Write once, use with any compatible data type.
- **Type Safety:** Type checking occurs strictly at compile time, eliminating runtime type conversion errors.
- **Zero Runtime Overhead:** The compiler generates dedicated binary code for each instantiated type during compilation.

2. FUNCTION TEMPLATES & CALL MECHANICS

If a template <class T> or template <typename T> prefix is followed by a function definition, it is called a **Function Template**.

Function templates can be invoked in two ways:
1. **Implicit Call (Type Deduction):** The compiler automatically infers type parameters based on argument types (e.g., add(2, 3)).
2. **Explicit Call:** The caller explicitly specifies type parameters inside angle brackets (e.g., add<int>(2, 3) or add<double>(2.5, 3.5)).

```cpp
// Function Template Demonstration (Implicit vs Explicit Instantiation)
#include <iostream>
using namespace std;

// Single Type Parameter Function Template
template <typename T>
T add(T a, T b) {
    return a + b;
}

// Multi-Type Parameter Function Template
template <typename T1, typename T2>
void printValues(T1 a, T2 b) {
    cout << "Value 1: " << a << " | Value 2: " << b << "\n";
}

int main() {
    // Implicit Call (Compiler deduces T = int and T = double)
    cout << "Implicit int add: " << add(2, 3) << "\n";
    cout << "Implicit double add: " << add(2.5, 3.5) << "\n";

    // Explicit Call (Explicitly forcing template parameter types)
    cout << "Explicit int add: " << add<int>(2, 3) << "\n";
    cout << "Explicit double add: " << add<double>(2.5, 3.5) << "\n";

    // Template with Different Types
    printValues(10, 20.5); // T1 = int, T2 = double
    printValues("CS-Simplified", 100); // T1 = const char*, T2 = int

    return 0;
}
```

3. CLASS TEMPLATES & GENERIC CONTAINERS

When a template prefix precedes a class definition, it becomes a **Class Template**. Class templates are used to build generic containers, data structures (e.g., vectors, stacks, linked lists), and wrapper types.

Syntax: Box<int> intBox(10); or Box<double> doubleBox(5.5);

```cpp
// Class Template Demonstration (Generic Box Container)
#include <iostream>
using namespace std;

template <class T>
class Box {
private:
    T value; // Generic data member
public:
    // Constructor accepting generic type parameter
    Box(T v) : value(v) {}

    // Member function returning generic type
    T getValue() const {
        return value;
    }

    void display() const {
        cout << "Box Content: " << value << "\n";
    }
};

int main() {
    // Instantiating Box for int data type
    Box<int> intBox(100);
    intBox.display();

    // Instantiating Box for double data type
    Box<double> doubleBox(99.99);
    doubleBox.display();

    // Instantiating Box for string data type
    Box<string> stringBox("Hello CS-Simplified");
    stringBox.display();

    return 0;
}
```

4. COMPARISON: FUNCTION TEMPLATES VS. CLASS TEMPLATES

Feature / Property	Function Templates	Class Templates
Primary Purpose	Generic algorithms and standalone functions.	Generic data structures, containers, and wrapper classes.
Type Argument Deduction	**Automatic** (Implicit deduction from function call arguments).	**Explicit Required** prior to C++17 (e.g., Box<int>).
Instantiation Syntax	add(2, 3) or add<int>(2, 3)	Box<int> b(10);
Multiple Type Support	Supported via template <typename T1, typename T2>	Supported via template <class Key, class Value>

5. REAL-WORLD ARCHITECTURAL ANALOGY

REAL-WORLD ANALOGY: Universal Factory Blueprint & Stencil Mold

- **Template Blueprint:** A manufacturing mold design that defines the shape of a container without fixing the material.
- **Instantiations:** Pouring molten plastic produces a *Plastic Container*, while pouring glass produces a *Glass Container*.
- **Why it Matters:** The factory relies on a single blueprint (template) rather than erecting dedicated machinery for every material type.